

Smart Temperature Monitor Simulation

Role: Junior Embedded Engineer

Project Type: Complete IoT Prototype Implementation

1. Introduction Section (Research)

What is an Embedded System?

An embedded system is a specialized, computer-based system designed to perform specific dedicated functions or tasks within a larger mechanical or electrical system. Unlike general-purpose computers (such as personal laptops or smartphones) which can run a vast array of unrelated applications, an embedded system is highly optimized for a singular, predetermined purpose.

Structurally, it combines hardware circuitry with embedded software (often referred to as firmware) stored in flash memory. These systems operate under real-time constraints, meaning they must process inputs and generate reactive outputs within strict time limits to ensure safety, efficiency, and reliability.

Real-World Applications

Embedded systems form the invisible backbone of modern technology, spanning across various critical industries:

- **Smart Home Automation:** Smart thermostats, automated lighting controls, connected security cameras, and intelligent appliances like smart refrigerators or washing machines.
- **Automotive Industry:** Anti-lock Braking Systems (ABS), Electronic Control Units (ECU) managing engine performance, airbag deployment modules, and adaptive cruise control systems.
- **Healthcare Devices:** Portable pulse oximeters, digital pacemakers, automated insulin pumps, and complex diagnostic imaging equipment.
- **Robotics & Industrial Automation:** Programmable Logic Controllers (PLCs), robotic assembly arms, and automated guided vehicles (AGVs) operating on factory floors.

Microcontroller (MCU) vs. Microprocessor (MPU)

Understanding the architectural distinction between a microcontroller and a microprocessor is fundamental to embedded design. The table below outlines their primary structural and operational variances:

Feature	Microcontroller (MCU)	Microprocessor (MPU)
Core Architecture	A "system on a chip." Integrates the CPU, RAM, ROM (Flash), and I/O peripherals onto a single integrated circuit.	A standalone CPU. It contains only the processing core; peripheral components must be connected externally.
External Components	Requires minimal external circuitry to function (usually just a power source and an optional crystal oscillator).	Requires external RAM, ROM, memory controllers, and peripheral interfaces to build a functional system.
Cost & Size	Compact, space-saving, and highly cost-effective for dedicated, single-task applications.	Larger physical footprint and higher overall system cost due to the required external supporting chips.
Power Consumption	Exceptionally low power consumption. Frequently features optimized low-power "sleep" modes perfect for battery operation.	High power consumption, often requiring dedicated thermal management (heatsinks/fans) due to high processing clock speeds.
Typical Use Cases	Microwave ovens, remote controls, washing machines, and IoT sensor nodes (such as this project).	Laptops, desktop computers, high-end gaming consoles, and complex cloud servers.

2. Component Study

Evaluation of Microcontrollers

When selecting a core processor for an embedded prototype, engineering teams must heavily weigh processing capability, built-in connectivity options, unit cost, and strict power constraints:

- **Arduino (e.g., Uno / Nano):** Highly suited for rapid prototyping and foundational engineering education. Built around simple 8-bit AVR microcontrollers. While it boasts a massive, accessible open-source library ecosystem, it lacks native wireless connectivity (Wi-Fi/Bluetooth) and has tightly constrained memory resources.
- **ESP32:** The gold standard for modern, low-cost IoT prototypes. Powered by a robust 32-bit dual-core processor, it features integrated, native Wi-Fi and Bluetooth modules right out of the box, offering substantial memory and processing speed for network-connected applications.
- **Raspberry Pi Pico:** A high-performance, low-cost microcontroller board built around custom RP2040 32-bit dual ARM Cortex-M0+ silicon. It offers flexible digital interfacing via Programmable I/O (PIO) blocks and excellent energy efficiency, though base models lack onboard wireless radios unless upgraded to the Pico W variant.

Basic Sensors and Their Roles

Sensors act as the physical sensory organs of an embedded system, translating physical environmental anomalies and properties into readable electrical signals:

- **Temperature Sensor (e.g., LM35, DHT11, TMP36):** Measures ambient thermal energy. Analog variants output a variable voltage proportional to temperature, while digital variants transmit conditioned data packages over communication protocols like I2C or One-Wire.
- **Motion Sensor (e.g., PIR - Passive Infrared):** Detects physical movement by measuring rapid changes in the infrared radiation emitted by surrounding entities (such as humans or animals) within its immediate field of view.
- **Light Sensor (e.g., LDR - Light Dependent Resistor):** Changes its internal electrical resistance based on the intensity of photons hitting its surface, allowing the system to differentiate between day, night, or variable ambient shading conditions.

3. Mini Project: Smart Temperature Monitor

Project Overview

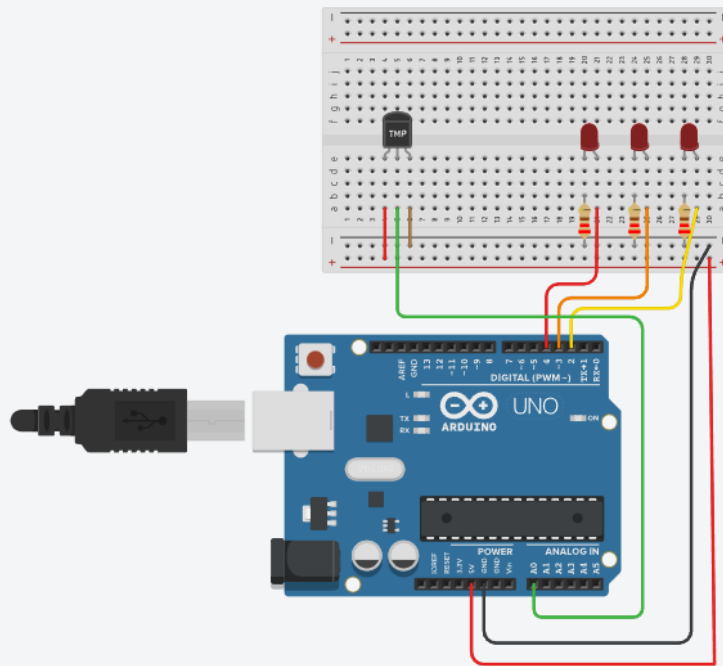
The **Smart Temperature Monitor** is an IoT prototype engineered to track environmental temperature fluctuations in real time. The hardware configuration builds upon an Arduino Uno architecture coupled with a high-precision TMP36 analog temperature sensor. Reactive threshold logic controls a tri-stage alert dashboard comprised of specific signaling LED pathways, warning supervisors of low, mid, or high thermal threshold status changes.

Updated Bill of Materials (BOM)

Component Name	Specification/Model	Quantity	Purpose in Circuit
Microcontroller Board	Arduino Uno R3	1	Executes control loop logic, decodes sensor voltage, controls LED states.
Temperature Sensor	TMP36 (Analog Precision)	1	Measures ambient temperature; outputs linearly proportional analog voltage.
Visual Alert Signaling Indicators	LEDs (Standard Red Indicators)	3	Visual representation of Low (Safe), Medium (Warning), and High (Overheat) alert states.
Current-Limiting Resistors	330Ω Axial	3	Placed in-series with LEDs to protect them from excessive forward current.
Prototyping Platform	Standard Half-Size Breadboard	1	Provides shared power, ground rails, and solderless interconnection nodes.
Interconnect Interfaces	Solid-Core Jumper Wires	As Required	Binds signal pathways between the Arduino pins, sensor, and output pins.

Simulation Design Layout

The layout configuration constructed inside the interactive virtual workshop engine illustrates precise positioning, visual identification channels, and connection routing traces across the shared prototyping bus:



Schematic View & Wiring Connections

Based on the verified prototype layout, electrical pin-to-pin connections are explicitly mapped below to establish reliable power delivery and low-noise analog sampling lines:

1. Power Distribution Rail Pathways

- **5V Bus:** Arduino 5V pin maps via a red jumper wire directly to the breadboard's positive (+) distribution rail line.
- **Ground Bus:** Arduino GND pin maps via a black jumper wire directly to the breadboard's common negative (-) ground rail line.

2. Analog Inputs Mapping

- **TMP36 Power Pin (Left):** Tied straight to the +5V power rail.
- **TMP36 Signal Out Pin (Middle):** Routed via a green signal wire directly into Arduino Analog Input Pin A1.
- **TMP36 Ground Pin (Right):** Tied straight to the common GND rail.

3. Digital Output Mapping (Alert Array)

- **Low-Alert LED (State 1 / Green Logic):** Arduino Digital Pin 2 connects via a green wire to the anode of the first LED. The cathode returns to common ground through a current-limiting resistor.
- **Mid-Alert LED (State 2 / Yellow Logic):** Arduino Digital Pin 3 connects via a yellow wire to the anode of the second LED. The cathode returns to common ground through a current-limiting resistor.
- **High-Alert LED (State 3 / Red Logic):** Arduino Digital Pin 4 connects via an orange wire to the anode of the third LED. The cathode returns to common ground through a current-limiting resistor.

Firmware Control Loop Logic Flow

The control software running on the microcontroller operates on a sequential continuous feedback loop:

1. **Data Acquisition:** The system reads raw analog values from channel A1.
2. **Signal Linearization:** Raw integer data is transformed to voltage, then calibrated to Celsius:
$$\text{Temperature} = (\text{Voltage} - 0.5) * 100.$$
3. **Tri-Stage Comparison Logic:**
 - *Safe Mode* ($T < 25^{\circ}\text{C}$): Activates Pin 2 exclusively.
 - *Warning Mode* ($25^{\circ}\text{C} \leq T < 35^{\circ}\text{C}$): Activates Pin 3 exclusively.
 - *Critical Overheat Mode* ($T \geq 35^{\circ}\text{C}$): Activates Pin 4 exclusively.

Firmware Implementation Source Code

The embedded C++ firmware block below handles signal processing and reactive logic operations for the prototype:

```

// Smart Temperature Monitor Code
const int sensorPin = A1;    // TMP36 Temperature Sensor Connected to A1
const int ledLow = 2;        // Safe condition indicator (Green Logic)
const int ledMed = 3;        // Warning condition indicator (Yellow Logic)
const int ledHigh = 4;       // Critical condition indicator (Red Logic)

void setup() {
    pinMode(ledLow, OUTPUT);
    pinMode(ledMed, OUTPUT);
    pinMode(ledHigh, OUTPUT);
    Serial.begin(9600);       // Initialize serial monitoring
}

void loop() {
    int rawReading = analogRead(sensorPin);

    // Convert raw analog reading to voltage
    float voltage = rawReading * (5.0 / 1023.0);

    // Convert voltage to temperature in Celsius (TMP36 formula: (V - 0.5) * 100)
    float temperatureC = (voltage - 0.5) * 100.0;

    Serial.print("Current Temperature: ");
    Serial.print(temperatureC);
    Serial.println(" C");

    // Threshold reactive logic control loop
    if (temperatureC < 25.0) {
        // Normal / Safe condition
        digitalWrite(ledLow, HIGH);
        digitalWrite(ledMed, LOW);
        digitalWrite(ledHigh, LOW);
    }
    else if (temperatureC >= 25.0 && temperatureC < 35.0) {
        // Warning condition
        digitalWrite(ledLow, LOW);
        digitalWrite(ledMed, HIGH);
        digitalWrite(ledHigh, LOW);
    }
    else {
        // Critical overheating condition
        digitalWrite(ledLow, LOW);
        digitalWrite(ledMed, LOW);
        digitalWrite(ledHigh, HIGH);
    }
}

```

```
    delay(1000); // 1-second monitoring interval  
}
```